3DG ENGINE

Wizard Combat Game Creation Guide

Animated Character | Magical Staff | Fire and Arcane Spells | Behavior-Tree Enemy | Nav Mesh

An engine-accurate field guide for building a playable third-person wizard encounter using the current 3DG Engine editor, runtime components, behavior graph, blackboard, custom AI scripts, particles, audio, and navigation systems.

VERSION 1.1 | JULY 2026

WORKED EXAMPLE Uses the existing wizardscene assets and code patterns already included in the repository, then extends them with the editor-authored AI workflow.

What You Will Build

A small vertical slice with a clear gameplay loop

The result is a third-person arena encounter. The player moves an animated wizard, carries a staff attached to the right-hand bone, selects a spell, plays a casting action, emits a muzzle burst, and launches a damaging magic projectile. An animated enemy patrols on the baked nav mesh, detects the player, chases around static obstacles, and performs a timed melee attack when its behavior tree says the player is in range.

Final scene objects

Object	Required parts	Runtime role
Player	Animated model, Player Controller, Health, WizardCombat script	Movement, camera, casting, projectile ownership
WizardStaff	Staff model	Bone-attached weapon mesh
StaffMuzzle	Small transform, Particle System, Audio Source	Cast origin and muzzle flash
FireBoltPrototype	Emissive mesh and optional particle trail	Clone source for fire projectiles
MagicImpactFX	Particle Effect and Audio Source	Reusable hit burst moved to each strike
Enemy	Animated model, Health, Navigation Agent, behavior brain	Patrol, perception, chase, and attack
NavMeshBoundsVolume	Editor navigation volume	Defines the navigation bake region
Level geometry	Static non-trigger colliders	Walkable-area obstacles and gameplay collision

Gameplay flow

```
Input -> cast animation -> staff muzzle burst -> projectile spawn
      -> projectile movement / damage -> impact particles + audio

Enemy perception -> Behavior Tree Selector
  -> Attack branch when CanAttack decorator passes
  -> Chase branch with nav-mesh path and Repath service
  -> Patrol fallback
```

ENGINE REALITY The editor authors the scene, particles, navigation volume, and .btgraph brain. Editor Play now advances animation and automatically applies montage-style movement locking. Attachment and projectile systems still need to run once per frame; the required bridge is shown later.

Repository Starting Point

Existing assets and references

The repository already contains a wizard sample that demonstrates the most difficult parts of this game: importing a UE-style rig, attaching separate animation FBXs, correcting Z-up orientation, attaching a staff to a hand bone, firing an animation-timed fireball, rendering particle trails, playing spatial audio, and navigating around pillars.

Path	Use it for
wizardscene/assets/models/WizardSM.FBX	Wizard skeleton and skinned mesh
wizardscene/assets/anims/Idle01Anim.FBX	Idle animation
wizardscene/assets/anims/WalkForwardAnim.FBX	Walk animation
wizardscene/assets/anims/BattleRunForwardAnim.FBX	Run animation
wizardscene/assets/anims/Attack01Anim.FBX	Casting / attack action
wizardscene/assets/staffs/Staff01SM.FBX	Magic staff
wizardscene/assets/sounds/whoosh.wav	Cast sound
wizardscene/assets/sounds/impact.wav	Projectile impact
wizardscene/src/WizardSceneApp.cpp	Working code reference for rig orientation, staff socket, fireball, particles, audio, and nav mesh
ecsgame/src/EcsGameApp.cpp	Working Projectile, Health, Attachment, and system-order reference

Recommended content layout

```
Content/
  Characters/Wizard/
  Characters/Enemy/
  Weapons/Staff/
  Particles/FireCast.particle
  Particles/FireTrail.particle
  Particles/MagicImpact.particlefx
  Audio/whoosh.wav
  Audio/impact.wav
  AI/EnemyWizard.btgraph
  Scenes/WizardArena.scene
```

DO THIS FIRST Copy or import the sample assets into the project Content root so editor asset paths remain stable. Do not build gameplay references against temporary source paths.

1. Build the Arena and Navigation Volume

Create walkable geometry

1. Add a large Plane for the floor. Give it a static collider and keep it at the intended navigation floor height.
2. Add walls, pillars, stairs, or other obstacles. Give every blocking object a non-trigger collider.
3. Leave dynamic rigid bodies out of the navigation obstacle bake; the current builder ignores bodies with positive inverse mass.
4. Keep doorways wider than twice the current hard-coded agent radius of 0.4 units, plus comfortable clearance.

Add the Nav Mesh Bounds Volume

1. Open Add and choose Navigation > Nav Mesh Bounds Volume.
2. Move the volume over the playable arena and scale X/Z to cover every point the enemy may visit.
3. Set its bottom face to the navigation floor. The Inspector reports Navigation floor as position Y minus half the absolute Y scale.
4. Open Debug > Navigation and enable Show Walkable Areas, then choose Rebuild Navigation.
5. Confirm green polygons cover the intended floor and stop around static obstacles.

What the current bake uses

Input	Current behavior
Bounds volume	Defines XZ bake limits and floor height; excluded from runtime rendering and collision.
Static box/pyramid/stair colliders	Use their XZ half-extents as obstacles.
Sphere/capsule/cylinder/cone colliders	Use radius-based XZ footprints.
Torus collider	Uses major radius plus minor radius as its footprint.
Cell size	0.5 units in the editor bake.
Agent radius	0.4 units in the editor bake.

APPLY ON NEXT PLAY In World Settings > Scene Debug, enable AI: Use Navmesh. This switches behavior-tree Chase and Move To Target from NavGrid to the funnel-smoothed NavMesh on the next Play session.

2. Create the Animated Player

Import and orient the wizard

1. Drag WizardSM.FBX into the scene and rename the object Player.
2. Enable Skeletal Model in the Animation Inspector and verify that the skeleton and clips can be inspected.
3. If the model faces downward, apply the engine's established UE-style correction: rotate -90 degrees around X. Then tune yaw so visual forward matches the engine's +Z gameplay forward.
4. Scale the mesh so its world height matches the Player Controller capsule; the wizardscene sample normalizes the model to roughly 1.8 units.

Add gameplay components

Component	Starting values
Player Controller	Third person; walk 4, run 7, jump 5, radius 0.4, height 1.8, camera distance 5
Health	100 / 100, Alive enabled
Native Script	WizardCombatScript
Collider / Rigid Body	Do not add them for the player controller; Play mode removes both and uses its own kinematic capsule

Player controls already supplied by the editor

- WASD moves relative to the camera.
- Shift sprints and Space jumps.
- Right mouse drag or pinned mouse-look rotates the view.
- V toggles first- and third-person view.
- The controller writes the Speed animation parameter from current movement input.

ORIENTATION CHECK Test forward movement before authoring cast direction. A projectile uses the Player transform's forward vector; a visually reversed character usually means the imported-model correction and gameplay transform have been mixed together.

3. Configure Locomotion and Casting Animation

Locomotion

1. In the selected Player's Animation section, enable Use Locomotion.
2. Assign Idle01Anim to Idle, WalkForwardAnim to Walk, and BattleRunForwardAnim to Run.
3. Start with Walk At = 0.15 and Run At = 3.0. Adjust only after Play-mode speed values are correct.
4. Use the Animation panel's Action Test to confirm Attack01Anim blends correctly over locomotion.

Create an action profile

1. Open Action Profiles and add a profile named StaffCast.
2. Choose Attack01Anim as the clip.
3. Choose an upper-spine Mask Root so the cast is layered over locomotion and the player can keep moving.
4. Start with Fade In 0.08, Fade Out 0.15, and Speed 1.0.

Montage-style movement lock

One-shot actions now control movement like Unreal Engine montages. A full-body action has an empty bone mask and blocks player movement, jumping, AI steering, and crowd displacement until the action finishes. A masked action affects only its chosen bone branch and leaves locomotion enabled. Camera look and gravity continue while a full-body action is locked.

Action setup	Movement behavior	Good uses
Empty Mask Root	Locked until the action clip completes	Melee attack, heavy hit reaction, knockdown, full-body death
Upper-spine Mask Root	Movement remains enabled	Staff cast, aim, reload, one-handed spell

Native Script movement-lock query

```
PlayAnimationProfile("StaffCast"); // masked profile: movement continues

PlayAnimationAction("HeavyAttack"); // empty mask: movement locks
if (IsAnimationMovementLocked()) {
    // Optional gameplay gate for input, abilities, or UI.
}
```

LOCK RELEASE The lock ends automatically when UpdateAnimations advances the full-body action to completion. Do not use a zero action speed, and do not omit the animation update from a custom runtime loop.

Animation events and current action-profile limitation

The engine supports authored animation events on base controller clips and exposes them to gameplay scripts through WasAnimationEvent. However, PlayAnimationProfile currently starts an action layer without copying the authored event list into that action. Therefore, use one of these two supported patterns:

Pattern	When to use
Action Profile + castDelay timer	Best current editor workflow; preserves upper-body layering and launches the spell after a tuned delay.
State Graph + Cast trigger + Cast.Release event	Use when frame-exact release is more important than an upper-body action layer.

REFERENCE wizardscene computes the release at roughly 40% of the attack clip and supplies an AnimEvent directly to AnimatedModel::PlayAction. The guide's script uses a configurable castDelay to reproduce that timing with editor-authored Action Profiles.

4. Attach the Magical Staff

Scene objects

1. Import Staff01SM.FBX and name the object WizardStaff.
2. Create a small object named StaffMuzzle near the staff tip. Add the cast Particle System and whoosh Audio Source to it.
3. The WizardCombat script will add Attachment components at runtime: the staff follows a right-hand bone and the muzzle follows the staff with a local offset.

Bone lookup and grip setup

Runtime bone-name fallback

```
static int FindRightHand(const engine::AnimatedModel& animated) {
    if (!animated.model) return -1;
    const auto& skeleton = animated.model->GetSkeleton();
    for (const char* name : {
            "hand_r", "RightHand", "hand_R", "Hand_R",
            "mixamorig:RightHand" }) {
        const int index = skeleton.Find(name);
        if (index >= 0) return index;
    }
    return -1;
}
```

Attachment values

An Attachment stores the parent entity, a local offset matrix, and an optional bone index. With boneIndex >= 0, UpdateAttachments combines the character transform, current skinned pose, inverse bone offset, and grip matrix. Tune translation, rotation, and scale until the staff sits naturally in the hand.

```
engine::Attachment staffSocket;
staffSocket.parent = Self();
staffSocket.boneIndex = handIndex;
staffSocket.offset =
    glm::translate(glm::mat4(1.0f), gripPosition) *
    glm::mat4_cast(glm::quat(gripEulerRadians)) *
    glm::scale(glm::mat4(1.0f), glm::vec3(gripScale));
Registry()->Add<engine::Attachment>(staffEntity, staffSocket);
```

SYSTEM ORDER Call UpdateAnimations before UpdateAttachments. The attachment needs the current frame's bone pose; reversing the order makes the staff lag by one frame or use stale pose data.

5. Author Fire and Other Magic

Recommended effects

Effect	Particle design	Blend / renderer
FireCast	Short orange/yellow sphere burst, outward velocity, 0.2-0.5 s lifetime	Additive billboard
FireTrail	Small continuous rate, short life, drag, upward force, red fade	Additive billboard
MagicImpact	Layered flash + sparks + smoke; 0 continuous rate, burst only	Additive flash/sparks + alpha smoke
IceShard	Blue/cyan narrow cone or mesh particles, low spread	Alpha or additive mesh
ArcaneBolt	Purple core, rotating motes, emissive mesh	Additive billboard + mesh

FireCast starting settings

Area	Suggested values
Playback	Autoplay off, loop off, local space on, max 150-300
Emission	Rate 0, burst 30-50, lifetime 0.20-0.45
Shape	Sphere radius 0.08-0.15 or cone 25-40 degrees
Color	HDR yellow/orange start to dark red transparent end
Size	0.20 start to 0.02 end

Projectile prototypes

Create FireBoltPrototype, IceBoltPrototype, and ArcaneBoltPrototype as ordinary scene objects below the floor but still inside the runtime scene. Give each an emissive mesh and optional Particle System with Autoplay off. The casting script clones the selected prototype and adds an engine::Projectile component.

DO NOT HIDE THE PROTOTYPE Runtime scene export skips invisible editor objects. Keep prototypes visible but place them below the floor, or create them directly in your standalone game code.

6. WizardCombatScript - Setup and Input

Attach this gameplay script to Player. It caches scene objects, installs staff/muzzle attachments, selects spells with 1/2/3, plays the StaffCast action on left-click, bursts the muzzle effect, and launches the projectile after castDelay.

WizardCombatScript.h - first half

```
#pragma once
#include <engine/gameplay/Script.h>
#include <engine/gameplay/GameplayComponents.h>
#include <engine/animation/AnimatedModel.h>
#include <engine/graphics/ParticleSystem.h>
#include <engine/graphics/SkinnedModel.h>

#include <GLFW/glfw3.h>
#include <glm/gtc/matrix_transform.hpp>
#include <algorithm>

class WizardCombatScript final : public engine::Script {
public:
    void OnCreate() override {
        staff_ = FindObject(GetFieldString("staff", "WizardStaff"));
        muzzle_ = FindObject(GetFieldString("muzzle", "StaffMuzzle"));
        fire_ = FindObject(GetFieldString("fireePrototype", "FireBoltPrototype"));
        ice_ = FindObject(GetFieldString("icePrototype", "IceBoltPrototype"));
        arcane_ = FindObject(GetFieldString("arcanePrototype", "ArcaneBoltPrototype"));
        AttachStaffAndMuzzle();
    }

    void OnUpdate(float dt) override {
        cooldown_ = std::max(0.0f, cooldown_ - dt);
        if (WasKeyPressed(GLFW_KEY_1)) spell_ = 0;
        if (WasKeyPressed(GLFW_KEY_2)) spell_ = 1;
        if (WasKeyPressed(GLFW_KEY_3)) spell_ = 2;

        if (WasMouseButtonPressed(GLFW_MOUSE_BUTTON_LEFT)
            && cooldown_ <= 0.0f && !pendingCast_) {
            pendingCast_ = true;
            castTimer_ = GetFieldFloat("castDelay", 0.35f);
            cooldown_ = GetFieldFloat("castCooldown", 0.65f);
            PlayAnimationProfile("StaffCast");
            if (muzzle_ != engine::ecs::kNull) {
                BurstParticles(muzzle_, 36);
                PlayAudio(muzzle_, true);
            }
        }

        if (pendingCast_) {
            castTimer_ -= dt;
            if (castTimer_ <= 0.0f) {
                pendingCast_ = false;
                SpawnSelectedSpell();
            }
        }
    }
```

WizardCombatScript - Attachments and Projectile Spawn

WizardCombatScript.h - second half

```
private:
    void AttachStaffAndMuzzle() {
        auto* animated = TryGet<engine::AnimatedModel>();
        if (!animated || !animated->model || !Registry()) return;

        int hand = -1;
        for (const char* name : {"hand_r", "RightHand", "hand_R",
                                 "Hand_R", "mixamorig:RightHand"}) {
            hand = animated->model->GetSkeleton().Find(name);
            if (hand >= 0) break;
        }
        if (staff_ != engine::ecs::kNull && hand >= 0) {
            engine::Attachment a;
            a.parent = Self();
            a.boneIndex = hand;
            a.offset = glm::translate(glm::mat4(1.0f), glm::vec3(0.0f)) *
                       glm::scale(glm::mat4(1.0f), glm::vec3(1.0f));
            Registry()->Add<engine::Attachment>(staff_, a);
        }
        if (muzzle_ != engine::ecs::kNull && staff_ != engine::ecs::kNull) {
            engine::Attachment a;
            a.parent = staff_;
            a.offset = glm::translate(glm::mat4(1.0f),
                                      glm::vec3(0.0f, 0.0f, 1.2f));
            Registry()->Add<engine::Attachment>(muzzle_, a);
        }
    }

    void SpawnSelectedSpell() {
        const engine::ecs::Entity prototype =
            spell_ == 1 ? ice_ : (spell_ == 2 ? arcane_ : fire_);
        if (!Registry() || prototype == engine::ecs::kNull) return;

        const engine::ecs::Entity bolt = Registry()->Clone(prototype);
        auto* boltTransform = TryGet<engine::ecs::Transform>(bolt);
        const auto* muzzleTransform = TryGet<engine::ecs::Transform>(muzzle_);
        const auto* playerTransform = Transform();
        if (!boltTransform || !playerTransform) return;

        boltTransform->position = muzzleTransform
            ? muzzleTransform->position
            : playerTransform->position + glm::vec3(0.0f, 1.0f, 0.0f);
        glm::vec3 direction = playerTransform->rotation * glm::vec3(0, 0, 1);
        direction = glm::normalize(direction);

        engine::Projectile projectile;
        projectile.dir = direction;
        projectile.owner = Self();
        projectile.speed  = spell_ == 1 ? 15.0f : (spell_ == 2 ? 22.0f : 18.0f);
        projectile.range  = spell_ == 2 ? 25.0f : 30.0f;
        projectile.damage = spell_ == 1 ? 20.0f : (spell_ == 2 ? 45.0f : 30.0f);
        projectile.radius = spell_ == 1 ? 0.9f : 0.8f;
        Registry()->Add<engine::Projectile>(bolt, projectile);

        if (auto* particles = TryGet<engine::ParticleSystemComponent>(bolt)) {
            particles->emitter.Clear();
```

```
            particles->emitter.cfg = particles->config;
            particles->gpuEmitter.reset();
            particles->initialized = false;
            particles->playing = true;
            particles->elapsed = 0.0f;
        }
    }

    engine::ecs::Entity staff_ = engine::ecs::kNull;
    engine::ecs::Entity muzzle_ = engine::ecs::kNull;
    engine::ecs::Entity fire_ = engine::ecs::kNull;
    engine::ecs::Entity ice_ = engine::ecs::kNull;
    engine::ecs::Entity arcane_ = engine::ecs::kNull;
    int spell_ = 0;
    float cooldown_ = 0.0f, castTimer_ = 0.0f;
    bool pendingCast_ = false;
};
```

TUNE THE SOCKET The grip and muzzle offsets above are placeholders because imported staff axes differ. Copy the working tuning approach from wizardscene, then store the final offsets in the script or expose them as script fields.

7. Register the Player Script and Expose Fields

Registration

```
#include "WizardCombatScript.h"
#include <engine/gameplay/Script.h>
#include <memory>

void RegisterWizardCombatScript() {
    engine::ScriptRegistry::Instance().Register(
        "WizardCombatScript",
        [] { return std::make_unique<WizardCombatScript>(); });
}
```

Call RegisterWizardCombatScript() once during game/editor startup before entering Play mode, then attach the same class name in the Inspector.

Suggested Inspector script fields

Field	Type	Value
staff	String	WizardStaff
muzzle	String	StaffMuzzle
fireProtoype	String	FireBoltPrototype
iceProtoype	String	IceBoltPrototype
arcaneProtoype	String	ArcaneBoltPrototype
castDelay	Float	0.35
castCooldown	Float	0.65

Alternative: exact animation event

For event-exact release, transition the Player's base state graph into a Cast state with SetAnimationTrigger("Cast"), add an authored Cast.Release event at the desired clip time, and spawn only when WasAnimationEvent("Cast.Release") returns true. This avoids the current action-profile event limitation but changes the base animation state instead of using the layered action profile.

FAIL SAFELY Every FindObject result can be kNull. Keep the checks shown in the example so a renamed prototype or muzzle does not crash Play mode.

8. Wire the Required Runtime Systems

The engine implements UpdateAnimations, UpdateAttachments, UpdateProjectiles, and UpdateHealth as reusable systems. Editor Play already advances animations in both normal and single-step play, and uses AnimatedModel::BlocksMovement() to freeze full-body actions. A standalone runtime must preserve the same order. Bone attachments and projectiles still need their reusable systems called once per frame.

Recommended frame order

Game-runtime bridge

```
// 1. Input, player controller, scripts, and AI choose actions.
engine::UpdateScripts(registry, dt, &scriptInput, &runtimeAudio);
UpdateAI(dt);

// Full-body actions have an empty mask and block movement automatically.
// Masked/layered actions keep locomotion enabled.

// 2. Advance actions so locks finish and sockets receive the current pose.
engine::UpdateAnimations(registry, dt);

// 3. Move staff/muzzle from the current bone pose.
engine::UpdateAttachments(registry);

// 4. Move projectiles and collect strikes before health bookkeeping.
const std::vector<engine::ProjectileHit> hits =
    engine::UpdateProjectiles(registry, dt);

// 5. Process hit VFX/audio, then update alive/justDied.
ProcessMagicHits(hits);
engine::UpdateHealth(registry);
```

Impact reaction

```
void ProcessMagicHits(const std::vector<engine::ProjectileHit>& hits) {
    for (const engine::ProjectileHit& hit : hits) {
        if (auto* t = registry.TryGet<engine::ecs::Transform>(impactFx))
            t->position = hit.point;
        engine::BurstParticleSystem(registry, impactFx,
                                    hit.lethal ? 120 : 70);
        // impactSoundSource is an entity with an AudioSource component.
        runtimeAudio.Play(impactSoundSource, true);
    }
}
```

EDITOR INTEGRATION Editor Play already calls UpdateAnimations and enforces full-body movement locking. Add only the missing attachment and projectile integrations to its play update path, once per frame. Do not call a world-level system from every entity script.

9. Create the Enemy

Enemy components

1. Duplicate the wizard or import a separate animated enemy model. Rename it Enemy.
2. Enable Skeletal Model and configure Idle, Walk, Run, and Attack clips.
3. Add Health and set 80-150 HP depending on desired encounter length.
4. Add Navigation Agent from the component Add menu.
5. Choose Player as Chase Target. Start with Vision Range 12 and Vision Half-Angle 60 degrees.
6. Set Speed 3.2, Max Force 20, Reach Radius 1.8, and Repath 0.25 seconds.
7. Add two or more patrol points by moving Enemy and choosing Add Waypoint (object pos) at each location.

Animation state graph

State / parameter	Configuration
Speed (Float)	Drives Idle, Walk, and Run; CombatSenseService writes current agent velocity length.
Attack (Trigger)	Any State -> Attack when triggered; higher priority than locomotion transitions.
Attack -> Idle	Use exit time near 0.9 and a non-interrupting return transition.
Attack clip	Non-looping; match attackWindup to the visual contact frame.

Perception caveat

CanSee casts from an eye point offset upward and forward because the perception query has no explicit source-exclusion argument. Keep the enemy's observer collider reasonably small so the eye ray does not immediately hit its own geometry.

TARGETING Use an explicit Player chase target for this single-player game. Team-based auto-targeting only considers other Navigation Agents on different non-zero teams.

10. Design the Blackboard

Open the Behavior Graph panel with F11. Expand Blackboard and add these initial values. Keys should use plain identifiers without spaces; the live store persists for the Play session and is shared by every node and BtScript instance on that enemy.

Key	Type	Initial value	Writer / reader
hasTarget	Bool	false	CombatSenseService -> CanAttack
targetAlive	Bool	false	CombatSenseService -> CanAttack
alerted	Bool	false	Set Bool task after perception succeeds
distanceToTarget	Float	999	CombatSenseService -> debug / decorators
attackRange	Float	1.8	CanAttack and AnimatedMeleeAttack
attackDamage	Float	15	AnimatedMeleeAttack
attackWindup	Float	0.35	AnimatedMeleeAttack
attackDuration	Float	0.80	AnimatedMeleeAttack
lastSeenPosition	Vec3	0, 0, 0	CombatSenseService when target is visible

Typed access

```
c.blackboard.SetFloat("distanceToTarget", distance);
c.blackboard.SetBool("hasTarget", c.targetEntity != engine::ecs::kNull);

const float range = c.blackboard.GetFloat("attackRange", 1.8f);
const bool alive = c.blackboard.GetBool("targetAlive", false);
```

LIVE DEBUG During Play, the Behavior Graph panel displays the first running graph agent’s blackboard snapshot and colors node borders yellow for Running, green for Success, and red for Failure.

11. Build the Behavior Tree

Recommended graph

```
ROOT  Selector                          [Service: CombatSenseService 0.05 s]
 |
 +-- Script Task: AnimatedMeleeAttack  [Decorator: CanAttack]
 |                                     [Decorator: Cooldown 0.90 s]
 |
 +-- Sequence: Chase Branch
 |    +-- See Target?
 |    +-- Set Bool  alerted = true
 |    +-- Chase                      [Service: Repath 0.25 s]
 |
 +-- Patrol
```

Create it in the panel

1. Add a Selector and choose Set as Root.
2. Drag its bottom output pin to empty space and create Script Task. Select AnimatedMeleeAttack in Script Class.
3. On the attack node choose + Decorator > Script Decorator and select CanAttack; add Cooldown and set 0.90.
4. Create a Sequence as the second root child. Preserve this order: See Target?, Set Bool, Chase.
5. Set the Set Bool key to alerted and Value to 1.
6. On Chase choose + Service > Repath and set Interval to 0.25.
7. Add Patrol as the final Selector child.
8. On the root Selector choose + Service > Script Service, select CombatSenseService, and set Interval to 0.05.
9. Save as Content/AI/EnemyWizard.btgraph. The panel must report valid.

Why this ordering works

- Selector tries attack first, chase second, and patrol last.
- CanAttack blocks the attack branch until the target is visible, alive, and inside the blackboard range.
- Cooldown prevents damage every frame after the timed attack task succeeds.
- Repath refreshes the Chase path while that branch is active.
- CombatSenseService refreshes blackboard and animation Speed while the whole root branch is active.

12. CombatSenseService

Place this header in editor/btscripts, register it, rebuild, and select it as a Script Service attachment on the root Selector.

```
#pragma once
#include <engine/ai/BtScript.h>
#include <engine/ai/BehaviorGraph.h>
#include <engine/animation/AnimatedModel.h>
#include <engine/gameplay/GameplayComponents.h>

#include <glm/glm.hpp>

class CombatSenseService final : public engine::ai::BtScript {
public:
    engine::ai::BtStatus Tick(engine::ai::AgentContext& c,
                              float /*dt*/) override {
        const bool validTarget = c.registry
            && c.targetEntity != engine::ecs::kNull
            && c.registry->Valid(c.targetEntity);

        bool alive = false;
        float distance = 999.0f;
        if (validTarget) {
            distance = glm::length(c.targetPos - c.agent.position);
            const auto* health =
                c.registry->TryGet<engine::Health>(c.targetEntity);
            alive = !health || health->alive;
            if (c.seesTarget)
                c.blackboard.SetVec3("lastSeenPosition", c.targetPos);
        }

        c.blackboard.SetBool("hasTarget", validTarget);
        c.blackboard.SetBool("targetAlive", alive);
        c.blackboard.SetFloat("distanceToTarget", distance);

        if (c.registry) {
            if (auto* animated =
                    c.registry->TryGet<engine::AnimatedModel>(c.self)) {
                animated->controller.SetParameter(
                    "Speed", glm::length(c.agent.velocity));
            }
        }

        const glm::vec3 toTarget = c.targetPos - c.agent.position;
        if (validTarget && glm::dot(toTarget, toTarget) > 1e-6f)
            c.facing = glm::normalize(toTarget);
        return engine::ai::BtStatus::Success;
    }
};
```

SERVICE STATUS A service's return status is ignored. Its job is to perform background work at the attachment interval while the wrapped branch is active.

13. CanAttack Decorator

The decorator is the branch gate. It returns false when the target is invalid, dead, invisible, or outside the authored blackboard attack range.

```
#pragma once
#include <engine/ai/BtScript.h>
#include <engine/ai/BehaviorGraph.h>

class CanAttack final : public engine::ai::BtScript {
public:
    bool Check(engine::ai::AgentContext& c) override {
        const bool hasTarget =
            c.blackboard.GetBool("hasTarget", false);
        const bool targetAlive =
            c.blackboard.GetBool("targetAlive", false);
        const float distance =
            c.blackboard.GetFloat("distanceToTarget", 999.0f);
        const float range =
            c.blackboard.GetFloat("attackRange", c.reachRadius);

        return hasTarget && targetAlive && c.seesTarget
            && distance <= range;
    }
};
```

Range consistency

Set the Navigation Agent Reach Radius to the same value as blackboard attackRange. The built-in Attack task also checks c.reachRadius internally. Keeping both values aligned prevents a decorator from allowing an attack that the task then rejects.

Using built-in nodes instead

For a non-animated prototype, you can remove CanAttack and AnimatedMeleeAttack, add the built-in Attack task, set Damage, then attach Sees Target, Target Within, and Cooldown decorators. The custom scripts are used here because the requested enemy must animate and apply damage at a deliberate contact time.

DECORATOR ORDER Attachments are authored top-to-bottom and compiled as wrappers. Put cheap validity/range gates before expensive or stateful modifiers whenever possible.

14. AnimatedMeleeAttack Task

This task starts the enemy attack as a full-body one-shot action, holds the agent still, waits for the windup, applies damage once, and reports Success after the authored duration. Because the action uses an empty mask, the engine also blocks AI steering and crowd displacement until the clip completes. Cooldown begins only after Success.

```
#pragma once
#include <engine/ai/BtScript.h>
#include <engine/ai/BehaviorGraph.h>
#include <engine/animation/AnimatedModel.h>
#include <engine/gameplay/GameplayComponents.h>
#include <glm/glm.hpp>
class AnimatedMeleeAttack final : public engine::ai::BtScript {
public:
    void OnEnter(engine::ai::AgentContext& c) override {
        elapsed_ = 0.0f;
        damageApplied_ = false;
        c.steer = glm::vec3(0.0f);
        if (c.registry) {
            if (auto* animated =
                    c.registry->TryGet<engine::AnimatedModel>(c.self)) {
                // Resolve this once from the imported clip list in production.
                const int attackClip = FindAttackClip(*animated);
                if (attackClip >= 0) {
                    animated->PlayAction(attackClip, {}, {},
                                         0.08f, 0.12f, 1.0f);
                }
            }
        }
    }
    engine::ai::BtStatus Tick(engine::ai::AgentContext& c,
                              float dt) override {
        if (!c.registry || c.targetEntity == engine::ecs::kNull
            || !c.registry->Valid(c.targetEntity))
            return engine::ai::BtStatus::Failure;
        auto* targetHealth =
            c.registry->TryGet<engine::Health>(c.targetEntity);
        if (!targetHealth || !targetHealth->alive)
            return engine::ai::BtStatus::Failure;
        const float distance = glm::length(c.targetPos - c.agent.position);
        const float range = c.blackboard.GetFloat("attackRange", 1.8f);
        if (distance > range) return engine::ai::BtStatus::Failure;
        c.steer = glm::vec3(0.0f);
        elapsed_ += dt;
        const float windup = c.blackboard.GetFloat("attackWindup", 0.35f);
        const float duration = c.blackboard.GetFloat("attackDuration", 0.80f);
        if (!damageApplied_ && elapsed_ >= windup) {
            targetHealth->Damage(
                c.blackboard.GetFloat("attackDamage", 15.0f));
            damageApplied_ = true;
        }
        return elapsed_ >= duration
            ? engine::ai::BtStatus::Success
            : engine::ai::BtStatus::Running;
    }
private:
    float elapsed_ = 0.0f;
```

```
    bool damageApplied_ = false;
};
```

FindAttackClip is a small project helper that returns the imported non-looping Attack clip index. Passing empty event and mask lists to PlayAction makes the action full body. If you instead supply a spine mask, the enemy can continue navigating during the attack.

CONTACT FRAME Tune attackWindup while watching the attack clip. Damage should occur when the weapon or hand visually reaches the player, not when the branch first starts.

15. Register the Behavior-Tree Scripts

The editor already calls RegisterGameBtScripts() during startup. Add the three headers and factories to editor/src/GameBtScripts.cpp, then rebuild. Script names appear in the Behavior Graph pickers only after registration.

```
#include "CombatSenseService.h"
#include "CanAttack.h"
#include "AnimatedMeleeAttack.h"

void RegisterGameBtScripts() {
    auto& r = engine::ai::BtScriptRegistry::Instance();
    r.Register("CombatSenseService", [] {
        return std::make_unique<CombatSenseService>();
    });
    r.Register("CanAttack", [] {
        return std::make_unique<CanAttack>();
    });
    r.Register("AnimatedMeleeAttack", [] {
        return std::make_unique<AnimatedMeleeAttack>();
    });
}
```

Assign the brain

1. Select Enemy and open the Navigation Agent section.
2. Drag Content/AI/EnemyWizard.btgraph from Assets onto Drop .btgraph here.
3. Confirm the Inspector shows the brain path instead of Built-in patrol/chase/search.
4. Enable AI: Use Navmesh in World Settings and enter Play.

Expected Console messages

```
AI: 'Enemy' running behaviour tree Content/AI/EnemyWizard.btgraph
AI: 1 nav agent(s) active (navmesh)
```

SAFE FAILURE A missing script class or malformed graph binds to a failing node instead of crashing. Use the Console and the graph's live red borders to locate the failure.

16. Test and Debug the Complete Encounter

Navigation validation

- Show Walkable Areas and rebuild after changing the bounds volume or static colliders.
- Confirm the polygon count is greater than zero.
- Place Player and every patrol point inside green navigation areas.
- Check that doors and gaps are wider than agent clearance.
- Enable AI Agent Debug to inspect perception, paths, reach radius, and steering guides.

Behavior-tree validation

Situation	Expected active branch
Player not visible	Patrol Running
Player visible and farther than 1.8	Chase Running; Repath service ticks
Player visible and at 1.8 or closer	AnimatedMeleeAttack Running, then Success
Attack just completed	Cooldown blocks attack; selector briefly uses Chase
Player dead	CanAttack fails after Health updates targetAlive

Magic validation

- Left click starts one casting action and one pending cast, not one projectile per frame.
- The staff follows the right hand through idle, walk, run, and cast poses.
- The projectile begins at StaffMuzzle and travels along the Player transform’s gameplay forward.
- The owner is ignored by projectile collision, and the first Health target inside radius takes damage.
- Impact FX moves to hit.point before its burst is triggered.

DEBUG FIRST AGENT The Behavior Graph panel currently displays the first running graph agent. When testing several enemies, isolate one agent or order the scene so the intended agent is first.

17. Common Problems

Symptom	Most likely cause and repair
Wizard faces down	UE-style Z-up model needs -90 degrees around X; keep this visual correction separate from gameplay yaw.
Projectile travels sideways	The mesh forward axis and Player transform forward disagree. Tune model yaw offset; keep projectile direction based on the gameplay transform.
Staff stays at origin	No valid hand bone, missing Attachment, or UpdateAttachments is not called after UpdateAnimations.
Cast plays but no spell appears	Prototype or muzzle RuntimeName mismatch, invisible prototype omitted during export, or projectile system not integrated.
Fire trail shares old particles	Cloned ParticleSystem transient state was not reset; clear emitter and reset GPU/initialized state.
Enemy walks through obstacles	AI: Use Navmesh is off, nav mesh was not rebuilt, or obstacle lacks a static non-trigger collider.
Enemy never sees Player	Wrong Chase Target, Player outside vision cone/range, LOS blocked, or observer collider blocks the eye ray.
Enemy damages every frame	Attack task has no Cooldown or applies damage repeatedly while Running.
Enemy reaches player but Attack fails	attackRange and Navigation Agent Reach Radius do not match.
Enemy moves but animation is frozen	CombatSenseService is not writing Speed, or UpdateAnimations is not called in the runtime loop.
Character never unlocks after an action	A custom runtime is not calling UpdateAnimations, the action speed is zero, or the clip cannot reach its end.
Character should move while casting but stops	The cast profile has an empty Mask Root. Choose the upper-spine root to make it a layered action.
Behavior graph script picker empty	BtScript was not included and registered in GameBtScripts.cpp, or the editor was not rebuilt.

Runtime system checklist

```
[ ] UpdateScripts / player input
[ ] Behavior tree and navigation AI
[ ] UpdateAnimations (advances actions and releases movement locks)
[ ] UpdateAttachments
[ ] UpdateProjectiles and process hits
[ ] UpdateHealth
[ ] Runtime particle update/render
[ ] Runtime audio update/listener
```

18. Production Checklist

A final pass before calling the vertical slice complete

Player and spells

- Player capsule, mesh scale, camera height, and floor contact agree.
- Idle, walk, run, and cast blend without foot sliding or sudden orientation flips.
- StaffCast uses an upper-spine mask so locomotion remains enabled.
- Heavy full-body attacks and hit reactions lock movement until their action clips complete.
- Staff grip and muzzle remain correct throughout every action.
- Fire, ice, and arcane prototypes use distinct visuals, speed, damage, range, and sound.
- Cast input is edge-triggered and cooldown-controlled.
- Projectile owner, damage radius, range expiry, and hit effects are tested.

AI and navigation

- Nav Mesh Bounds Volume covers the entire playable arena and uses the correct floor height.
- Static colliders carve expected holes and the green overlay has no accidental islands.
- Enemy speed, force, reach radius, vision, and repath interval are tuned together.
- Blackboard values update live and every task/decorator/service is registered.
- Attack damage occurs once at the visual contact frame and Cooldown prevents frame-rate-dependent damage.
- Enemy full-body attacks resist navigation steering and crowd displacement until the action finishes.
- Target death and missing-target states fail safely.

Runtime and build

- Animation, attachment, projectile, health, particle, and audio systems run once in a deliberate order.
- The final game target copies assets beside the executable or resolves them from the project content root.
- The game has been tested after a clean CMake configure and rebuild, not only in an existing build folder.
- Console warnings, missing factories, asset load failures, and zero-polygon nav bakes have been cleared.